

Hough Transform

Ecole Centrale de Lyon

Option Informatique

Computer Vision

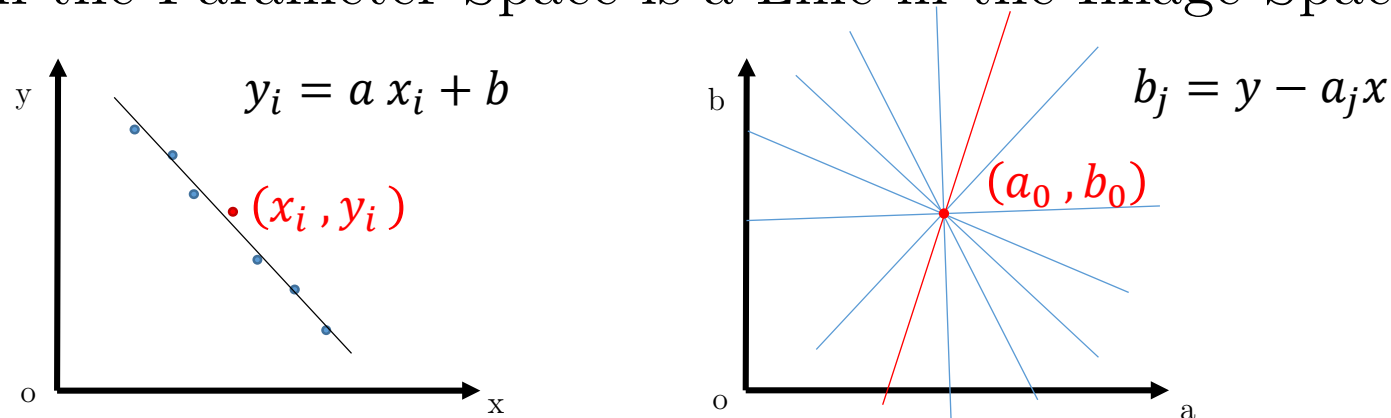
Mohsen ARDABILIAN

Hough Transform

- Direct Object Detection method
 - Elegant
 - Based on edge information
 - Edge points not need to be connected
 - Key Ideas
 - Duality
 - Edge Vote mechanism
 - Edges not need to be connected

Duality: Image and Parameters Spaces

- Line equation $y = ax + b$ in the Image Space
- Find (a_0, b_0) that satisfies $y_i = a_0 x_i + b_0$ for all (x_i, y_i)
- Line equation $b = y - ax$ in the Parameter Space
- Duality between Image and Parameters Spaces
 - A point in the Image Space is a Line in Parameter Space
 - A point in the Parameter Space is a Line in the Image Space



Hough Transform Algorithm

Consider the Parameter Space as an Accumulator

$Acc(j, k)$ initialized to zero

For each edge point (x_i, y_i) from the Image Space

For each parameters cell (a_j, b_k) : for j , for k

If the condition $b_k = y_i - a_j x_i$ is satisfied

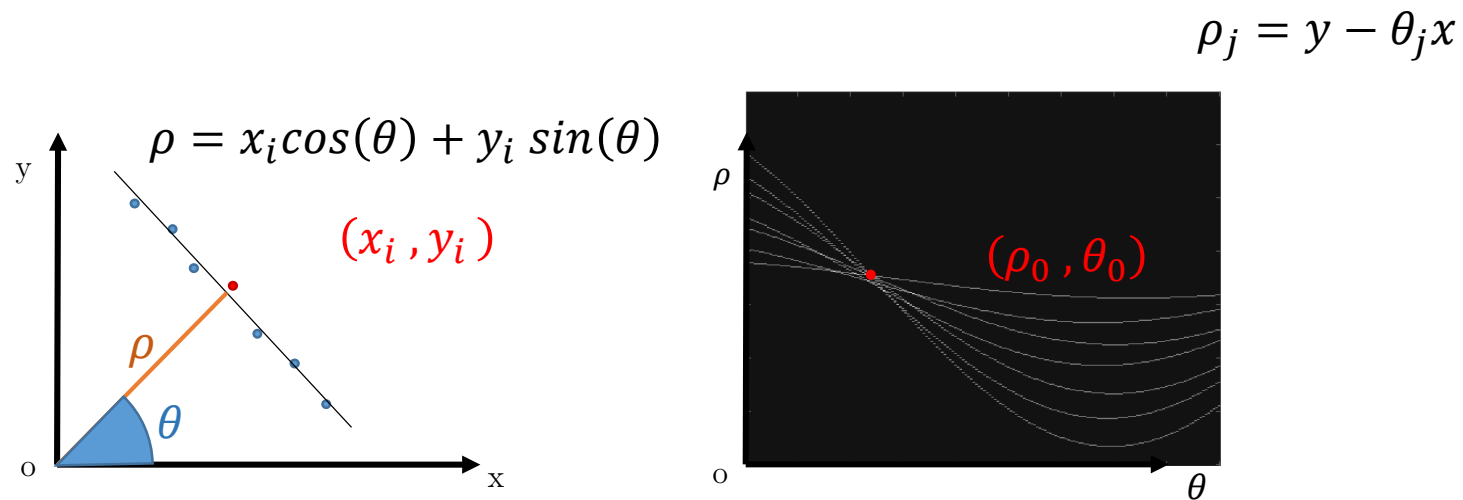
$Acc(j, k) += 1$

Acc thresholding and **max** detection

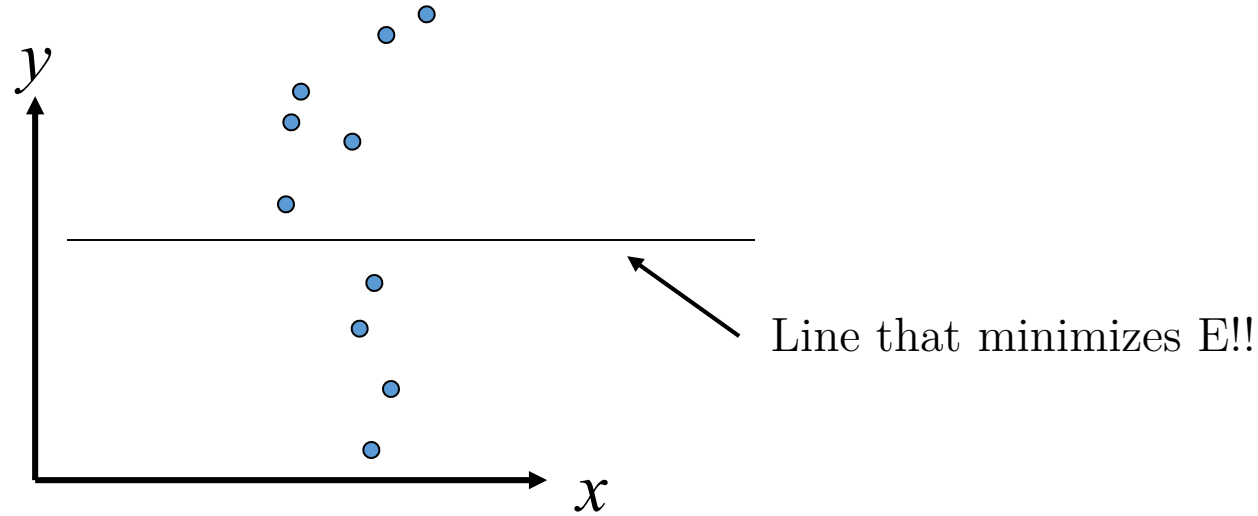
For each **max** verify line segments in the Image Space

Better Line Parametrization

$$\rho = x \cos(\theta) + y \sin(\theta)$$



Problem with Parameterization



Solution: Use a different parameterization

(same as the one we used in computing Minimum Moment of Inertia)

$$E = \frac{1}{N} \sum_i (\rho - x_i \cos \theta + y_i \sin \theta)^2$$

Note: Error E must be formulated carefully!

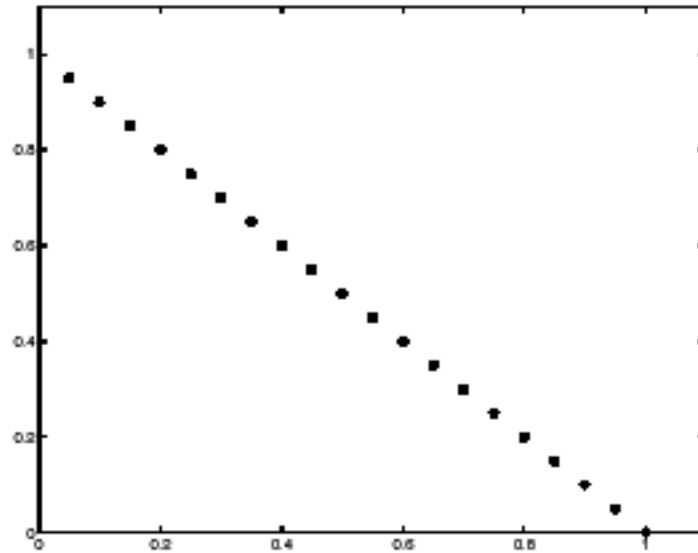
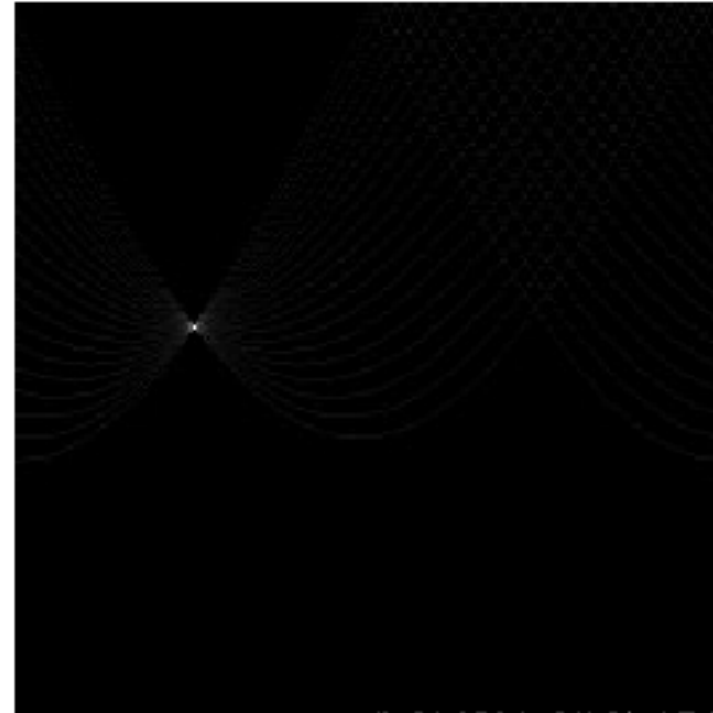


Image space



Votes

Horizontal axis is θ ,
vertical is ρ .

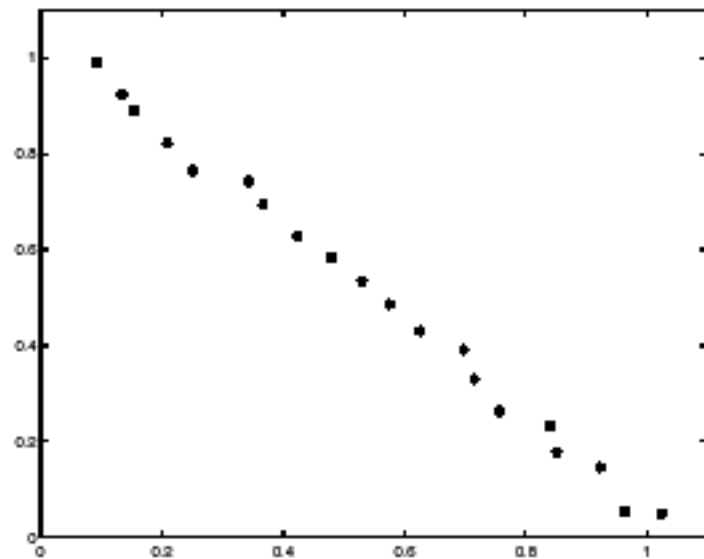
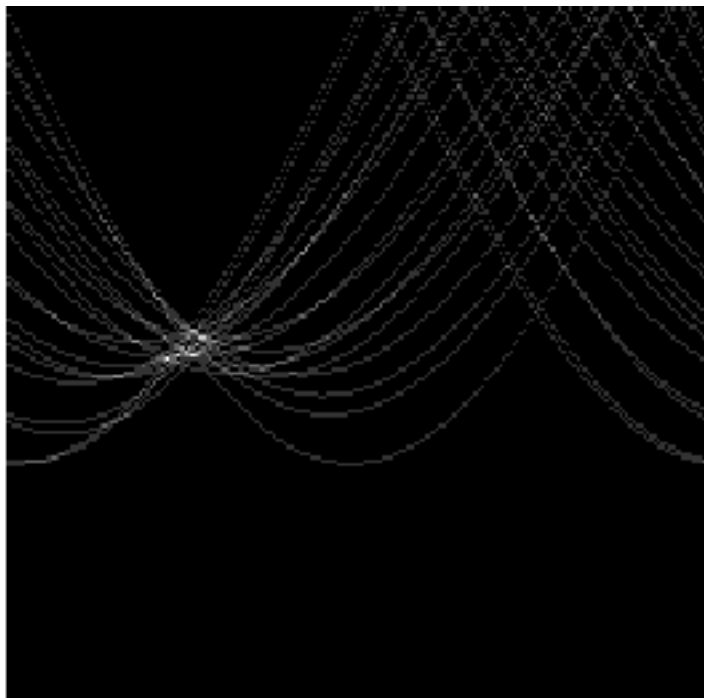
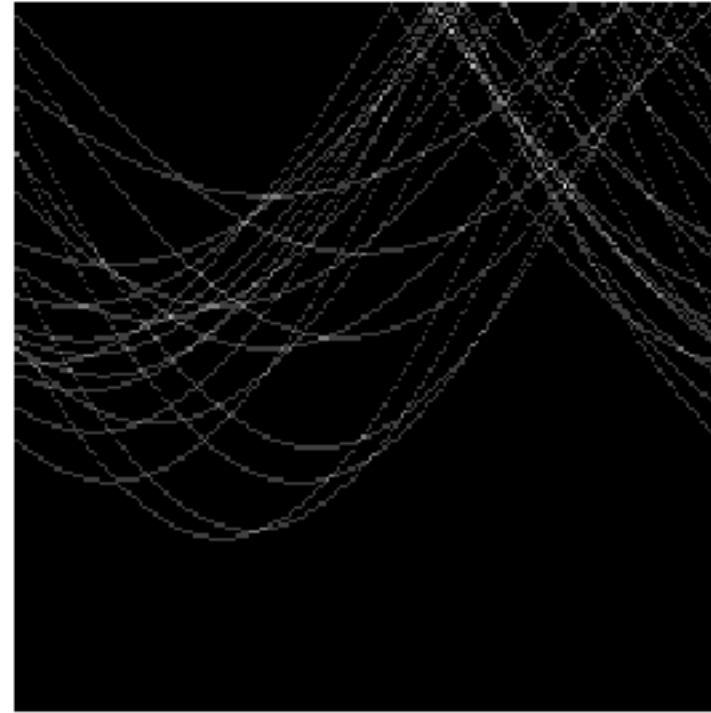
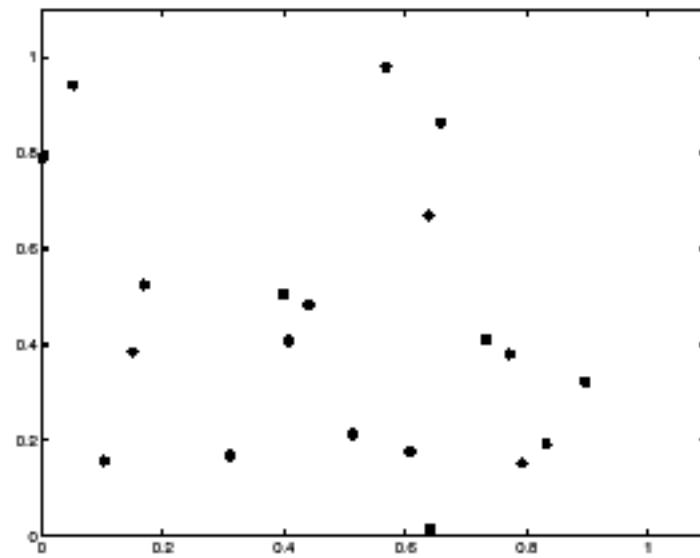


Image
space



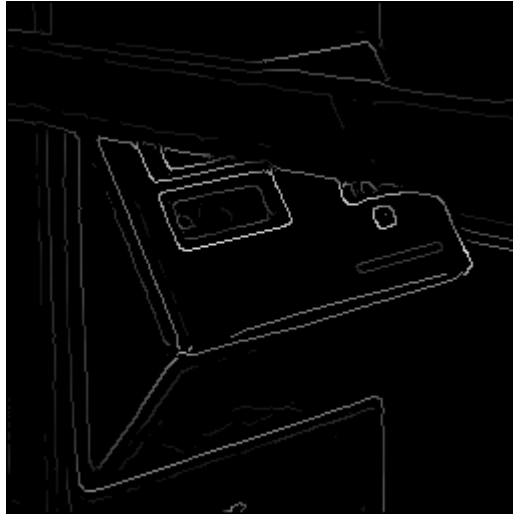
votes



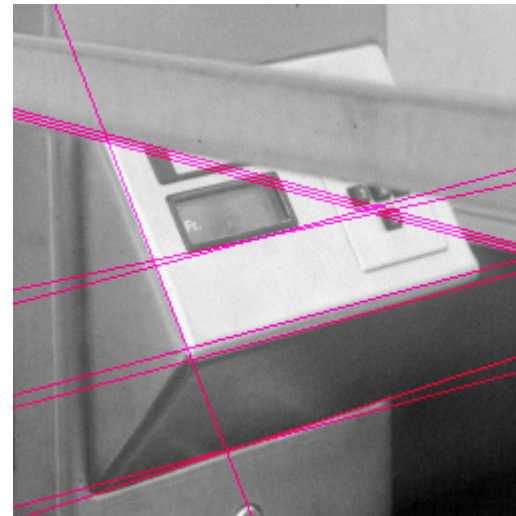
Real World Example



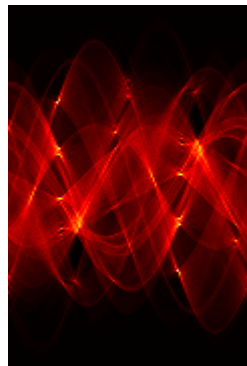
Original



Edge Detection



Found Lines

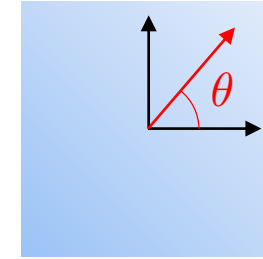


Parameter Space

Improvement

Extension 1: Use the image gradient

1. Initialize the Accumalator
2. for each edge point $I[x,y]$ in the image



$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

$$\theta = \text{gradient at } (x, y)$$

$$d = x \cos(\theta) - y \sin(\theta)$$

$$\theta = \tan^{-1} \left(\frac{\frac{\partial I}{\partial y}}{\frac{\partial I}{\partial x}} \right)$$

$$Acc[d, \theta] += 1$$

3. Find the value(s) of (d, θ) where $Acc[d, \theta]$ is maximum
4. The detected line in the image is given by

$$d = x \cos(\theta) - y \sin(\theta)$$

(Reduces degrees of freedom)

Circles by Hough Transform

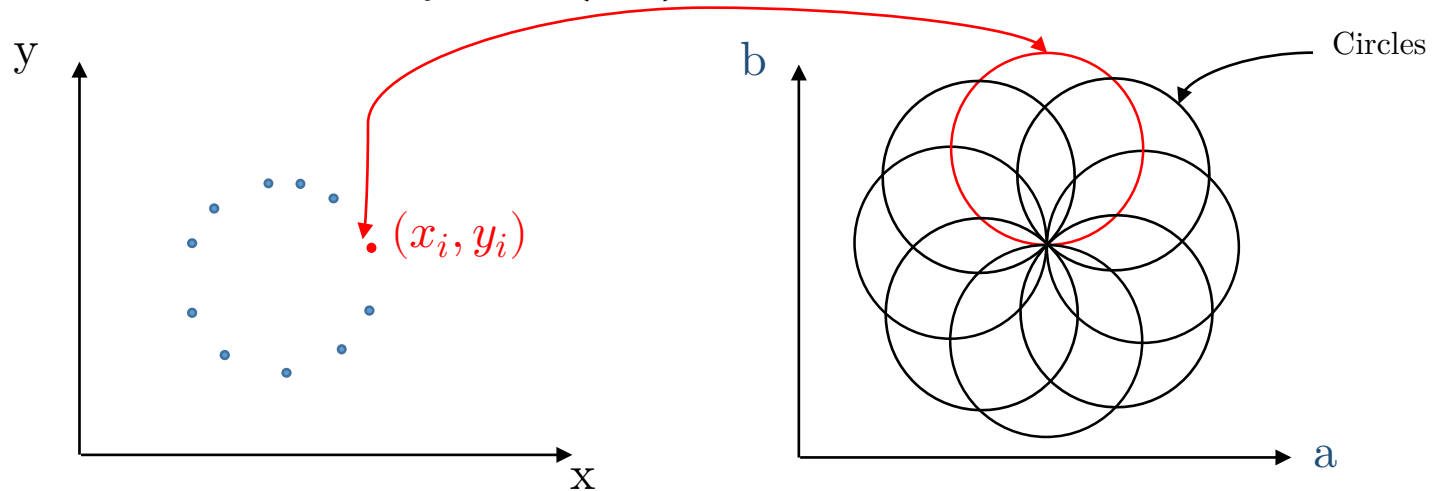
Equation of Circle:

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

For a known radius: (2D Hough Space)

Accumulator Array

Accumulator Array : $Acc(a, b)$



Finding Circles by Hough Transform

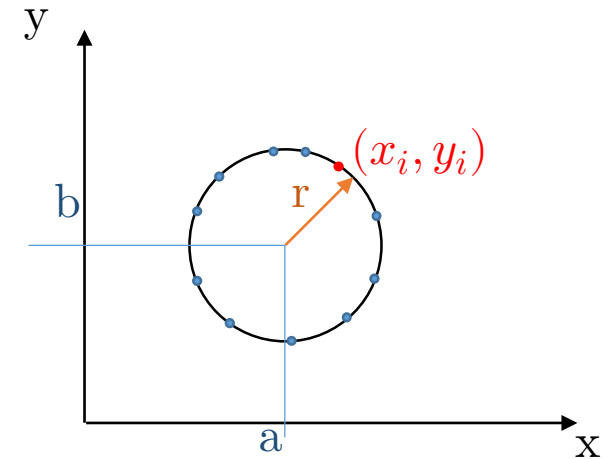
Equation of Circle:

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

For an unknown radius : 3D Hough Space!

Accumulator array

$$\text{Accumulator Array : } \text{Acc}(a, b, r)$$



What is the surface in the Hough space?

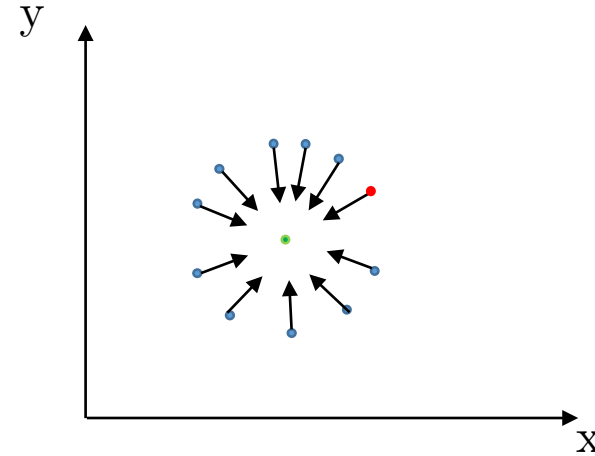
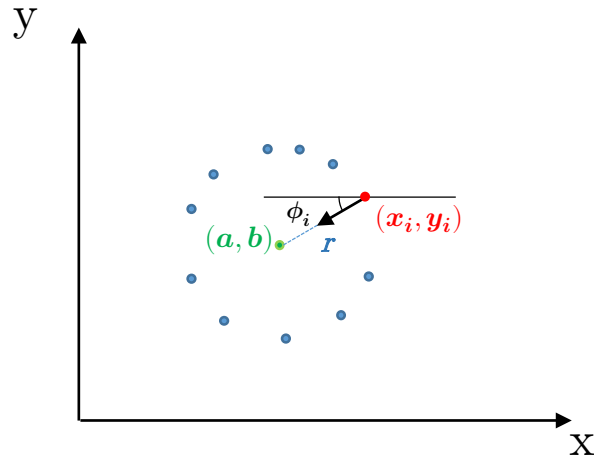
Using Gradient Information for Circle

Gradient information can save lot of computation:

Edge Location (x_i, y_i)

Edge Direction ϕ_i

Assume radius is known:



$$b = y - \sin \phi$$

$$a = x - \cos \phi$$

Need to increment only one point in Accumulator!!

Real World Circle Examples



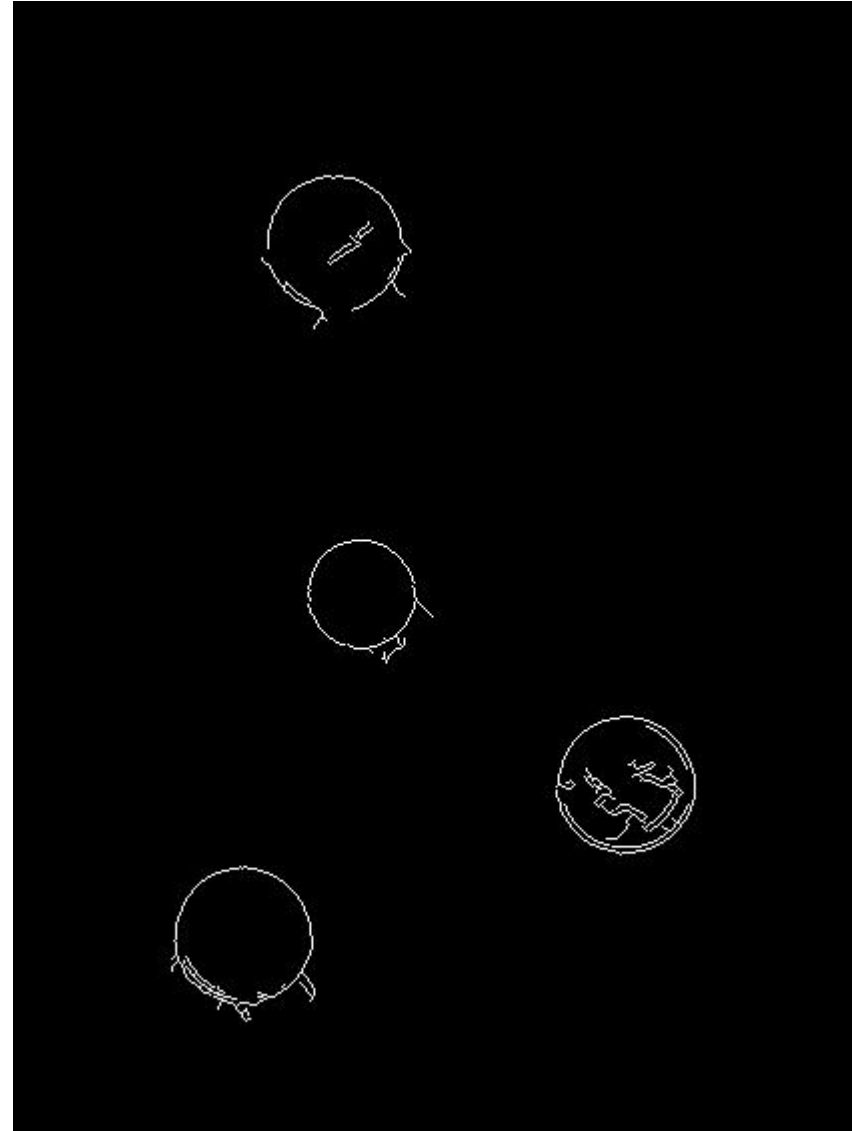
Crosshair indicates results of Hough transform, bounding box found via motion differencing.

Finding Coins

Original



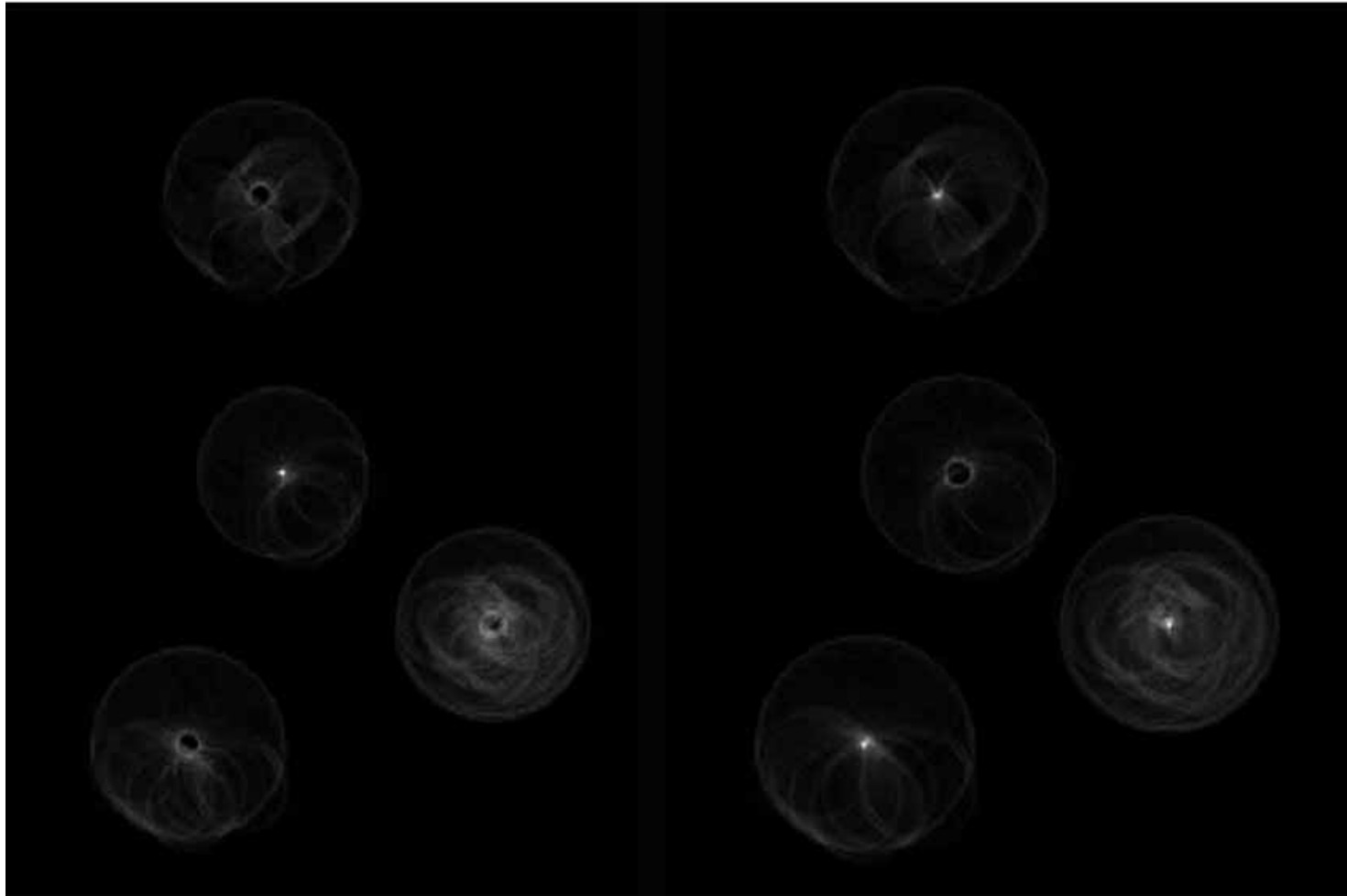
Edges (note noise)



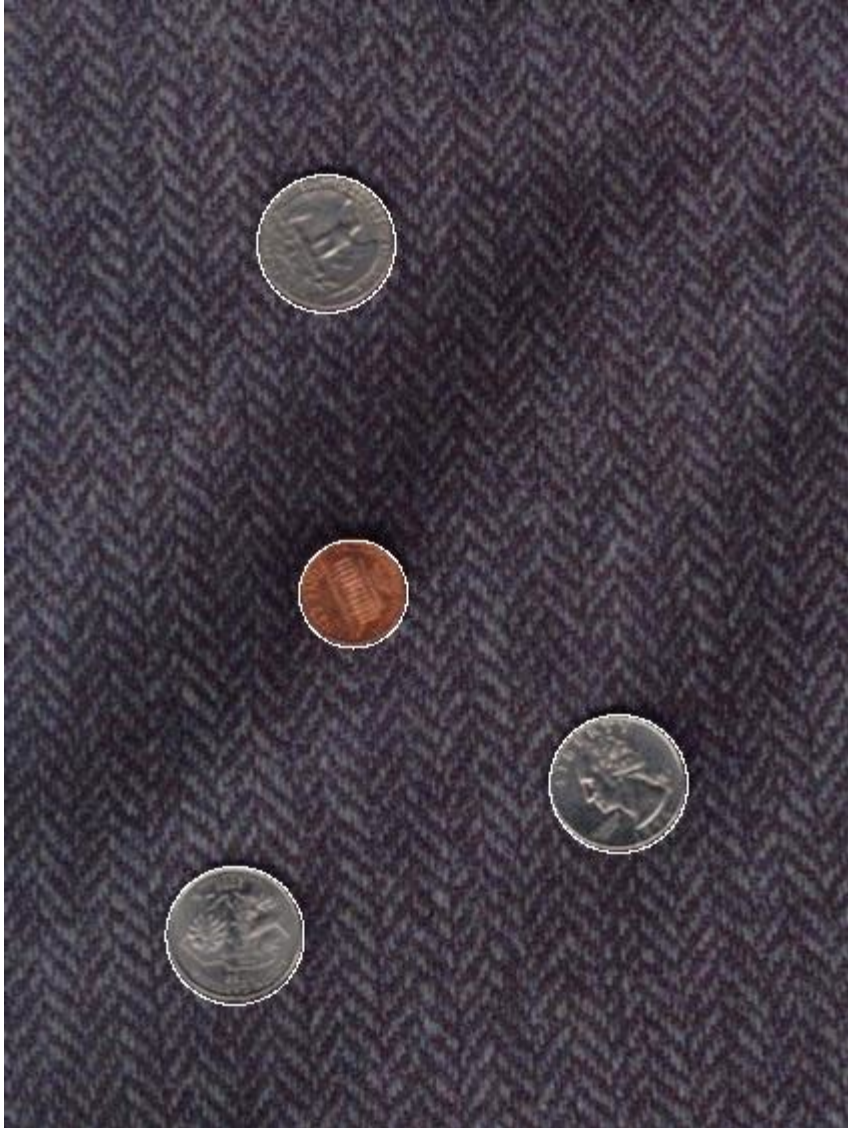
Finding Coins

Penny

Quarters



Finding Coins



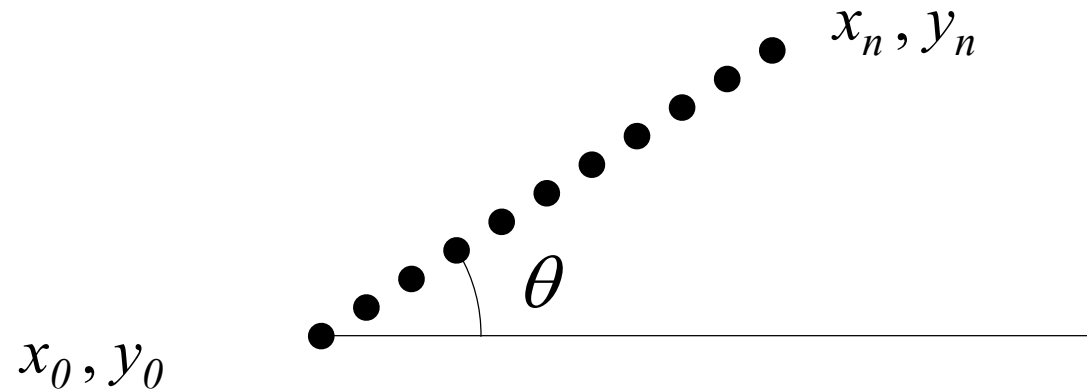
Note that because the quarters and penny are different sizes, a different Hough transform (with separate accumulators) was used for each circle size.

Coin finding sample images from: Vivek Kwatra

Fast Connective Hough Transform

Property:

x_i and y_i can be
defined
induction
(recurrence)



$$x_1 = x_0 + 1,$$

$$y_1 = E(y_0 + (x_1 - x_0) \tan(\theta)) = E(y_0 + \tan(\theta))$$

$$x_2 = x_0 + 2,$$

$$y_2 = E(y_0 + (x_2 - x_0) \tan(\theta)) = E(y_0 + 2 \tan(\theta))$$

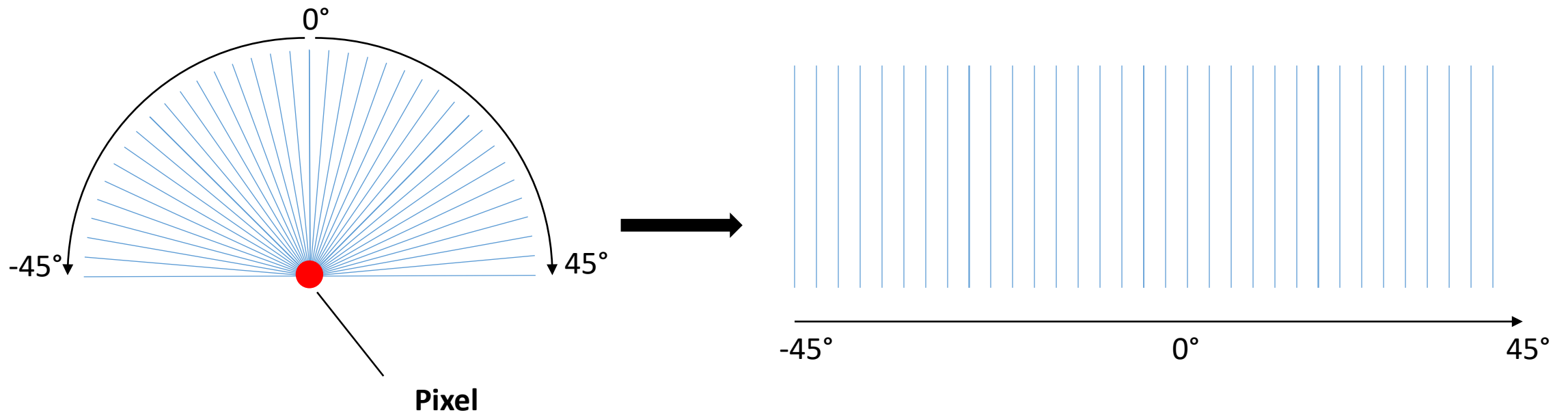
...

...

$$x_n = x_0 + n,$$

$$y_n = E(y_0 + (x_n - x_0) \tan(\theta)) = E(y_0 + n \tan(\theta))$$

Pixel Neighborhood Transformation

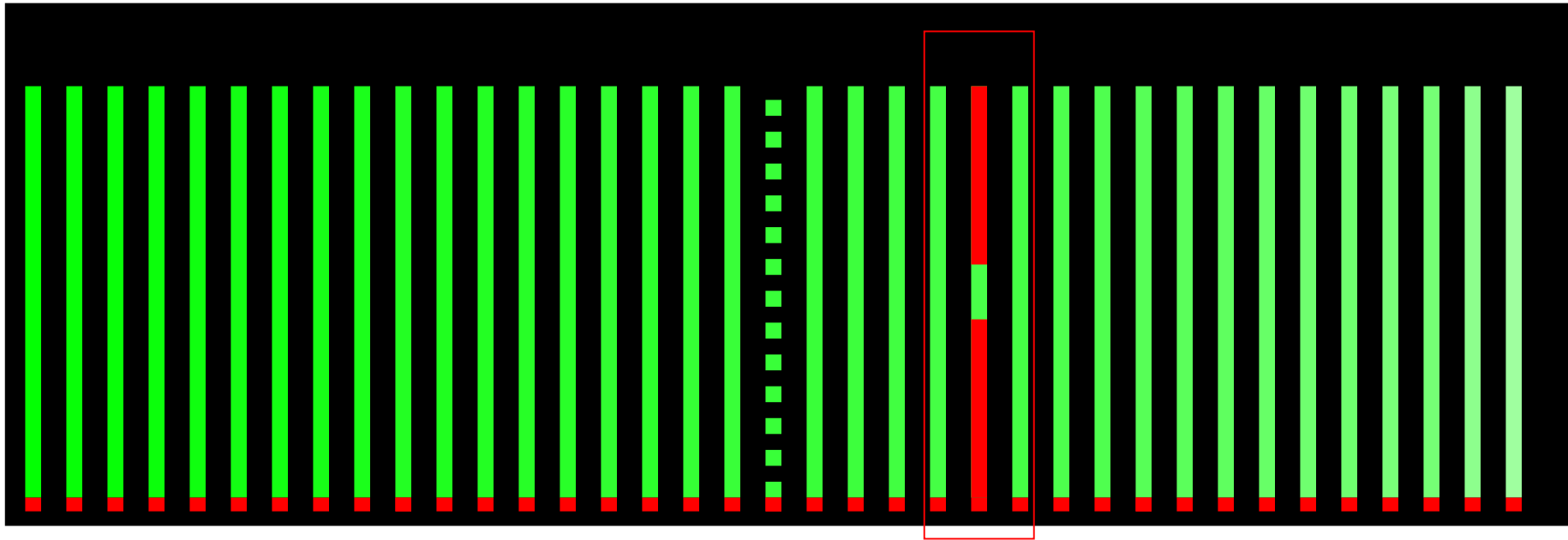


Fast Connective Hough Transform

$$X = \begin{pmatrix} n & n & . & . & . & n \\ . & . & & & . & . \\ . & . & & . & & . \\ . & . & . & & & . \\ 2 & 2 & . & . & . & 2 \\ 1 & 1 & . & . & . & 1 \end{pmatrix} + x_0 \mathbf{1}_{n,45} = TX_{n,45} + x_0 \mathbf{1}_{n,45}$$

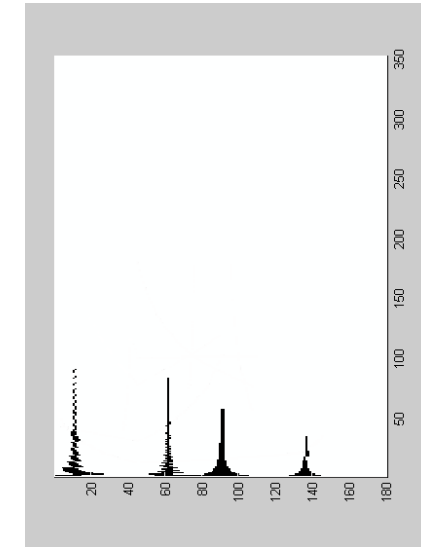
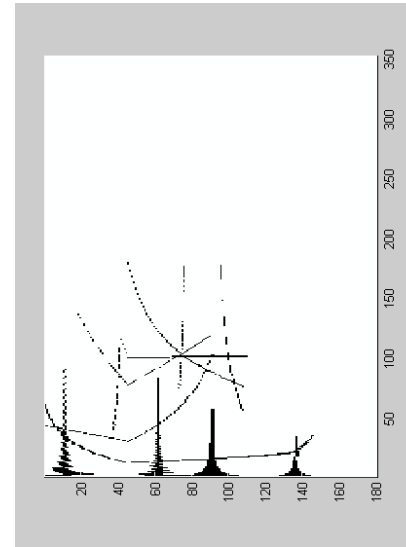
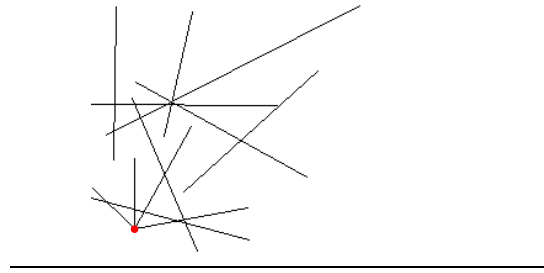
$$Y = \begin{pmatrix} n \tan 0 & n \tan 1 & . & . & . & n \tan 45 \\ . & . & & & . & . \\ . & . & & . & & . \\ . & . & . & & & . \\ 2 \tan 0 & 2 \tan 1 & . & . & . & 2 \tan 45 \\ \tan 0 & \tan 1 & . & . & . & \tan 45 \end{pmatrix} + y_0 \mathbf{1}_{n,45} = TY_{n,45} + y_0 \mathbf{1}_{n,45}$$

Fast Connective Hough Transform

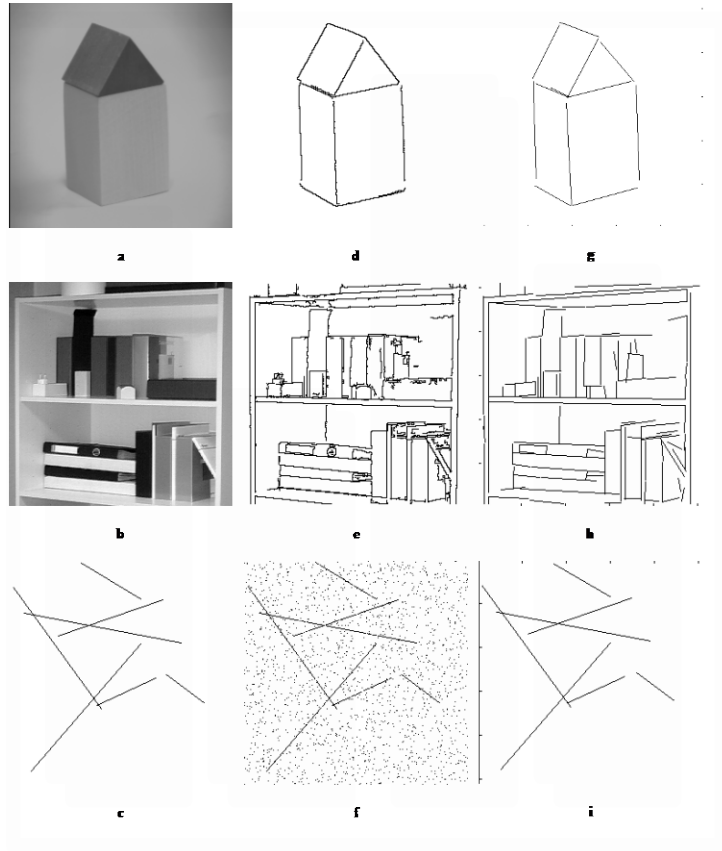


Fast Connective Hough Transform

- Move into the image
- Project the local neighborhood into the transform space (r, θ)
- Detect Segments
- Label pixels as Processed



Fast Connective Hough Transform



- Image
- Contours
- Extraction des segments par *FCHT*

	Nb pixels	Time in sec	Time in sec
		<i>RHT</i>	<i>FCHT</i>
Image a	1107	1,3625	0,061
Image b	1771740	38.29	1.8

- Comparison of the calculation time: *RHT* (Randomized Hough Transform) & *FCHT* (Fast Connective Hough Transform)
- A Ratio of 20!

Bibliography and literature to read

Computer Vision: Algorithms
and Applications
Richard Szeliski

4.3 Lines : pp 251

Ex 4.12: Hough transform line
detector

Questions & improvements

- What finesse for the discretization of the parameter space?
- What min, max, and center for parameters space?
- How to estimate line segments?
 - Connectivity
 - Discontinuities
- How to deal with the parameters space after each max detection?
- What about other shapes as circles, etc.
- Let's practice with Matlab!
 - First, Line Segments Detection

Compute and Display Hough Transform

Read an image, and convert it to a grayscale image.

```
%Compute and Display Hough Transform
%Read an image, and convert it to a grayscale image.
RGB = imread('gantrycrane.png');
I   = im2gray(RGB);
%Extract edges.
BW = edge(I, 'canny');
%Calculate Hough transform.
[H,T,R] = hough(BW, 'RhoResolution', 0.5, 'Theta', -90:0.5:89);
%Display the original image and the Hough matrix.
subplot(2,1,1);
imshow(RGB);
%Copyright 2012 The MathWorks, Inc.
```

Compute and Display Hough Transform

```
title('gantrycrane.png');  
subplot(2,1,2);  
imshow(imadjust(rescale(H)), 'XData', T, 'YData', R, 'InitialMagnification', 'fit');  
title('Hough transform of gantrycrane.png');  
xlabel('\theta'), ylabel('\rho');  
axis on, axis normal, hold on;  
colormap(gca, hot);
```

Detect Five Strongest Circles in an Image

This example shows how to find all circles in an image, and how to retain and display the strongest circles.

```
%Read a grayscale image into the workspace and display it.
A = imread('coins.png');
imshow(A)
%Find all the circles with radius r pixels in the range [15, 30].
[centers, radii, metric] = imfindcircles(A,[15 30]);%
%Retain the five strongest circles according to the metric values.
centersStrong5 = centers(1:5,:);
radiiStrong5 = radii(1:5);
metricStrong5 = metric(1:5);
%Draw the five strongest circle perimeters over the original image.
viscircles(centersStrong5, radiiStrong5, 'EdgeColor', 'b');
%Copyright 2012 The MathWorks, Inc.
```

Thank you for your
attention!

Practice & Report

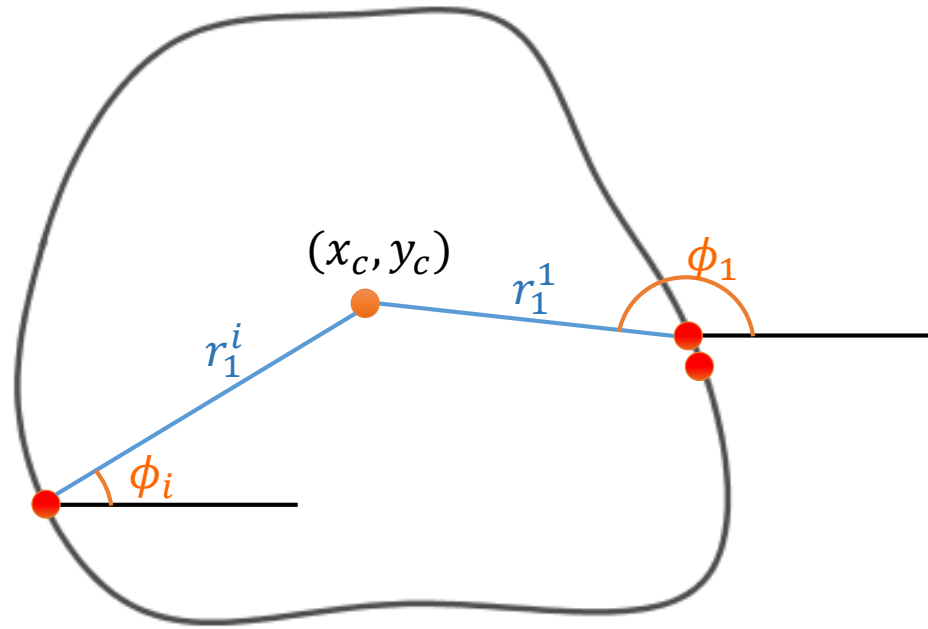
- Exercise with report (to be evaluated)
 - Computer Vision: Algorithms and Applications, Richard Szeliski
 - Page 251 Hough Transform
 - See Lane Departure Warning System.pdf
 - Make your synthesis and summary of this chapter (1 page)
 - Apply HT to one (some) image(s)
 - Comment on your work

Generalized Hough Transform

- Model Shape NOT described by equation

Generalized Hough Transform

- Model Shape NOT described by equation
- Shape Model Characterization in offline stage : ϕ Table



Generalized Hough Transform

- Model Shape NOT described by equation
- ϕ Table

Edge direction

$\bar{r}$

ϕ_1

$\bar{r}_1^1, \bar{r}_2^1, \bar{r}_3^1$

ϕ_2

$\vdots$

$\vdots$

ϕ_i

$\bar{r}_1^i, \bar{r}_2^i$

$\vdots$

$\vdots$

ϕ_n

$\bar{r}_1^n, \bar{r}_2^n$

Generalized Hough Transform

Find Object Center (x_c, y_c) given edges (x_i, y_i, ϕ_i)

Create Accumulator Array $A(x_c, y_c)$

Initialize: $A(x_c, y_c) = 0 \quad \forall (x_c, y_c)$

For each edge point (x_i, y_i, ϕ_i)

For each entry $\overline{r}_k^i$ in table, compute :

$$x_c = x_i + r_k^i \cos(\phi_i)$$

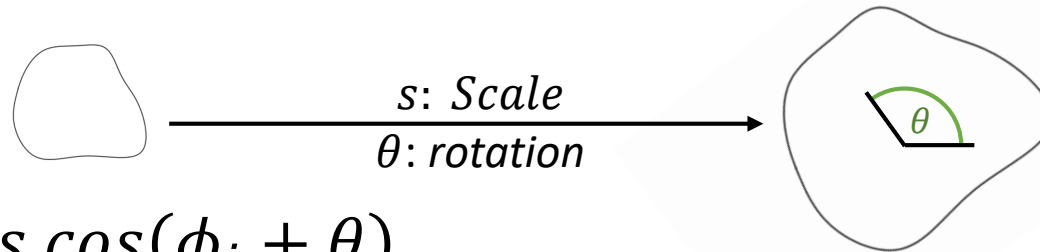
$$y_c = y_i + r_k^i \sin(\phi_i)$$

Increment Accumulator: $A(x_c, y_c) = A(x_c, y_c) + 1$

Find Local Maxima in $A(x_c, y_c)$

Generalized Hough Transform

- Rotation and Scale
 - Use accumulator array $A_c[x_c, y_c, s, \theta]$



- if $(x_c = x_i + r_k^i s \cos(\phi_i + \theta))$

and

$$y_c = y_i + r_k^i s \sin(\phi_i + \theta))$$

then

$$A_c[x_c, y_c, s, \theta] += 1$$

Hough Transform: Comments

- Works on Disconnected Edges
- Relatively insensitive to occlusion
- Effective for simple shapes (lines, circles, etc)
- Trade-off between work in Image Space and Parameter Space
- Handling inaccurate edge locations:
 - Increment Patch in Accumulator rather than a single point